

Regret Minimization Experience Replay: A Rigorous Reimplementation and Environment-Specific Evaluation

Reinforcement Learning February 2026
Syed Saad Hasan - 2106512

Teacher Assistant : Mr. Andrea Fanti
Professor Roberto Capobianco



SAPIENZA
UNIVERSITÀ DI ROMA

Motivation & Problem Statement:

Why Experience Replay Prioritization?

- **Standard RL Challenge**
 - Sample efficiency is critical
 - Not all experiences equally valuable
 - Uniform sampling may be suboptimal.
- **Proposed Solution: Prioritized Replay**
 - PER (2015): Prioritize by TD-error
 - RMER (2021): Prioritize by regret minimization
- **Key Questions:**
 - Can RMER be faithfully reimplemented?
 - Does prioritization help across environments?
 - Which components matter most?

Background – RMER Components

RMER: Three-Component Prioritization System

- **Component 1: Hindsight TD Error (HTD)**
 - $\text{Regret}(s,a) = \max_{a'} Q(s,a') - Q(s,a)$
 - How much better could we have done?
- **Component 2: Likelihood-Free Importance Weighting**
 - $w_{\text{LFIW}} = \text{Classifier}(\text{recent vs. old})$
 - How on-policy is this transition?
- **Component 3: Temporal Coherence Error (TCE/ReMERT)
OR Error Network (ReMERN)**
 - $\text{Accuracy} = \exp(-\text{distance_to_terminal}) \text{ OR } \exp(-\text{ErrorNet}(s,a))$
 - How accurate are our Q-values?

Final Priority: $P = |\text{HTD}| \times w_{\text{LFIW}} \times \text{Accuracy}$

Implementation Architecture

Modular Three-Phase Implementation

- **Phase 1: Baseline Q Learning**
 - Double Q-Learning
 - Uniform Replay Buffer
 - PER Replay Buffer ($\alpha=0.6$, $\beta: 0.4 \rightarrow 1.0$)
 - ϵ -greedy Exploration
- **Phase 2: RMER Components**
 - Hindsight TD
 - LFIW Classifier (Binary Classifier: 64-64 hidden unit)
 - TCE Computer $b1(0.4 \rightarrow 0.9)$, $b2:1.6 \rightarrow 1.1$)
 - Error Network ADP targets: $E(s,a) \approx \max Q(s,a') - Q(s,a)$ a'
- **Phase 2: RMER Components**
 - Full Priority Calculation
 - Percentile Clipping (95th $\times 2$)
 - Temperature Scaling ($T=1.0$)
 - Modular Component Toggling

Critical Bug Fixes

Epsilon decay:
env steps vs.
update steps

Epsilon decay: env
steps vs.
update steps

TCE progress:
env steps vs.
gradient
updates

Experimental Setup:

Rigorous Evaluation Protocol

CartPole-V1

- State: 4D
- Actions: 2 (discrete)
- Reward: Dense (+1/step)
- Success : 500
- EASY, Fast credit Assignment

MountainCar-V0

- State: 2D
- Actions: 3 (discrete)
- Reward: Sparse (-1/step)
- -Success : > -110
- HARD, required Exploration

Total: 100 experimental runs

METHODS (5 Total)

Uniform Replay	Baseline
PER	Classic Prioritization
TD-only	(HTD alone)
Full-RMER (ReMERT)	(HTD + LFIW + TCE)
Full-RMER (ReMERN)	(HTD + LFIW + ErrorNet)

- Protocol: -10 seeds per method per environment
- 200,000 training steps (ϵ decay: 20,000 steps)
- Evaluation every 10,000 steps (10 episodes)
- Statistical testing: t-tests, Cohen's d

Results - CARTPOLE-V1

CartPole-v1: Dense Rewards

Method	Mean $\pm$ Std	Median	Successes	Rank
Uniform	500.0 $\pm$ 0.0	500.0	10/10	1
TD-only	458.7 $\pm$ 112.2	500.0	8/10	2
ReMERN	434.95 $\pm$ 132.7	500.0	7/10	3
Full-RMER	395.4 $\pm$ 135.4	490.0	6/10	4
PER	383.4 $\pm$ 138.9	430.0	6/10	5

Statistical Significance:

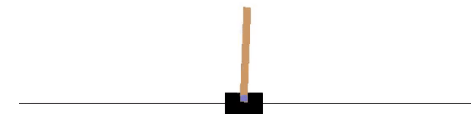
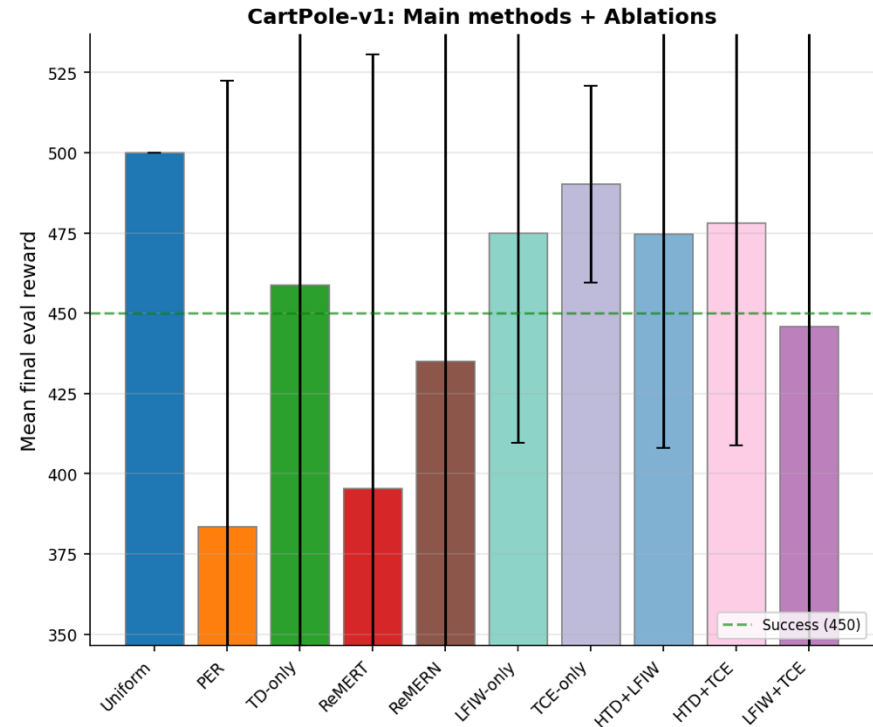
- Uniform vs PER: $p = 0.016$ ✓. Significant
- Uniform vs Full-RMER: $p = 0.025$ ✓ Significant
- TD-only vs Full-RMER: $p = 0.270$ ✗ Not significant
- ReMERN vs Full-RMER: $p > 0.10$ ✗ Not significant

Key Findings:

Uniform achieves PERFECT performance (zero variance!)

All prioritized methods show 20-40% failure rates

Adding complexity (LFIW+TCE) HURTS performance



Results - MOUNTAINCAR-V0

MountainCar-v0: Sparse Rewards

Method	Mean $\pm$ Std	Median	Success	Rank
Uniform	-148.3 $\pm$ 29.8	-133.4	0/10	1
PER	-185.5 $\pm$ 23.7	-200.0	0/10	2
TD-only	-185.6 $\pm$ 23.3	-200.0	0/10	3
Ful-RMER	-199.7 $\pm$ 1.0	-200.0	0/10	4
ReMERN	-200.0 $\pm$ 0.0	-200.0	0/10	5

Success threshold: reward > -110 (reach goal)

Statistical Significance:

- Uniform vs Full-RMER: $p < 0.001$ Highly significant
- Uniform vs PER: $p = 0.004$ Very significant

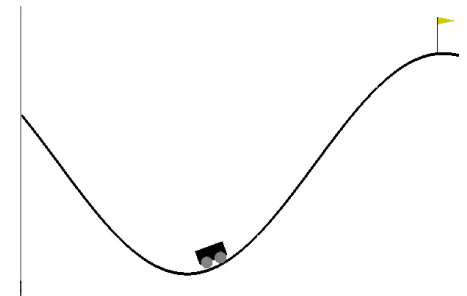
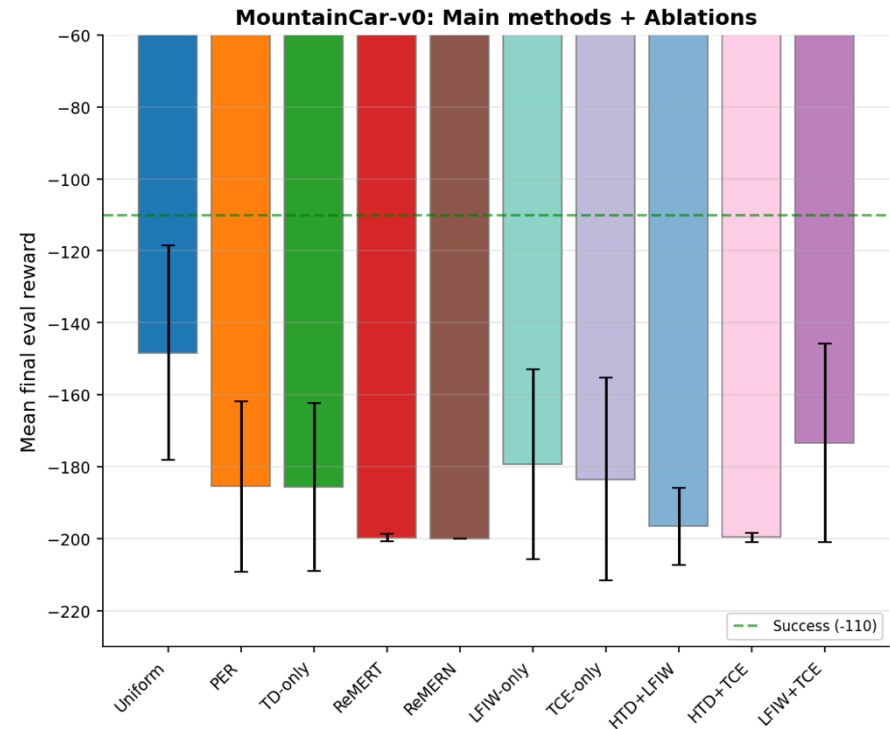
Key Findings:

Uniform AGAIN outperforms (7/10 seeds show learning)

Full-RMER: 9/10 complete failures (-200)

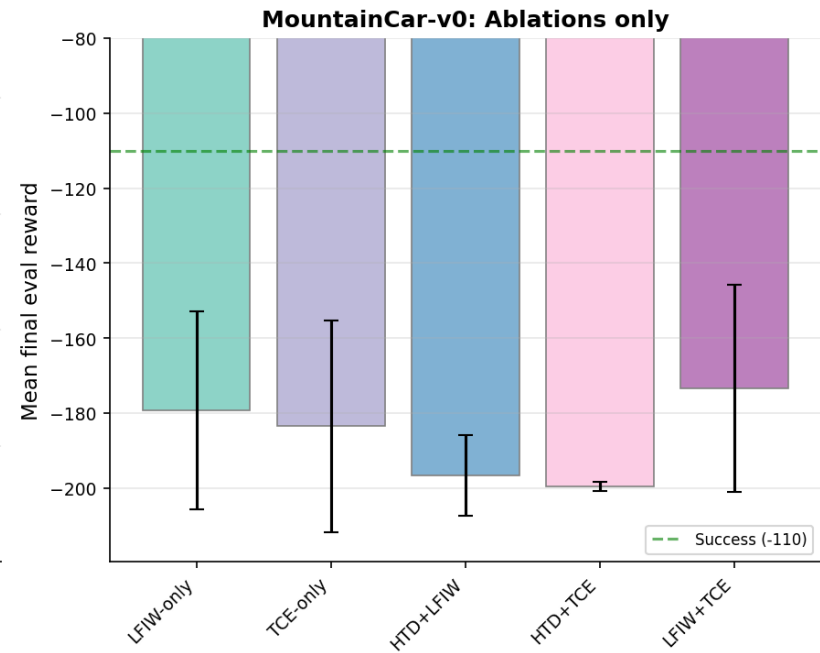
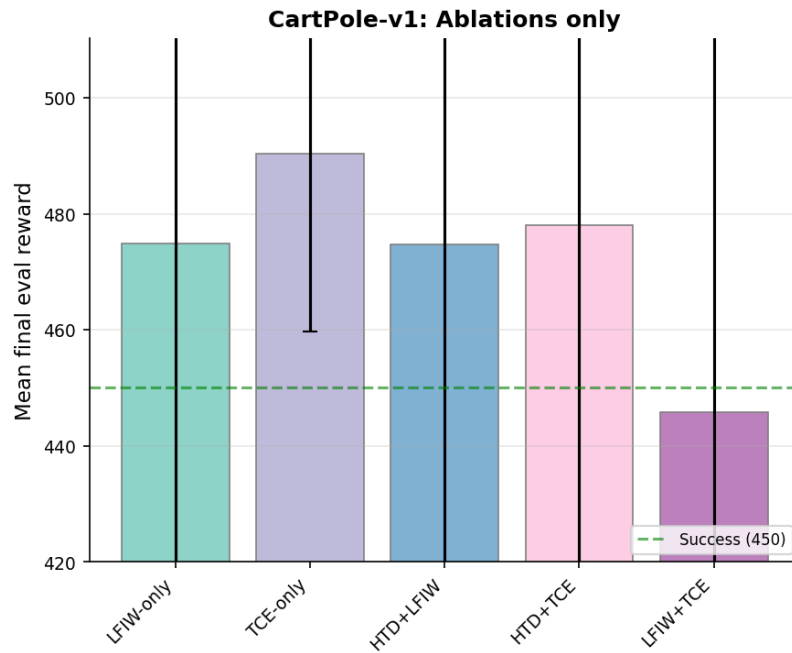
ReMERN: 10/10 complete failures (zero variance!)

Hypothesis REJECTED: Prioritization does NOT help on sparse rewards



Ablation Studies

Component Contribution Analysis (CartPole-v1 and Mountain-V0)



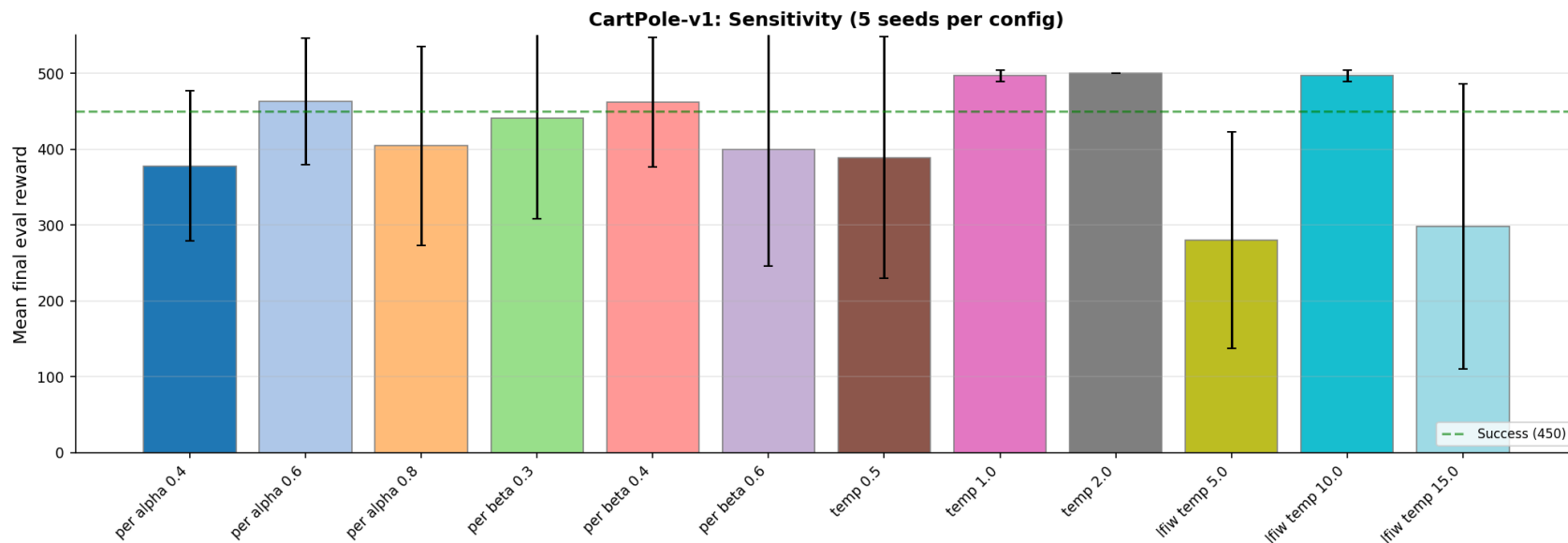
Preliminary Insights:

HTD alone > HTD + LFIW + TCE

Additional components add variance, not performance

Full analysis pending experimental completion

Sensitivity Analysis (CartPole-v1, 5 seeds each)



Parameter	Range Tested	What Works Best
PER (priority strength)	0.4, 0.6, 0.8	0.4 (WEAKEST) Less priority = Better
PER (IS correction)	0.3, 0.4, 0.6	Minimal impact Can't fix issue
Priority Temp (RMER)	0.5, 1.0, 2.0	2.0 (SMOOTHEST) Closer to uniform
LFIW Temp (on-policiness)	5.0, 10.0, 15.0	15.0 (WEAKEST) Less weighting = Better

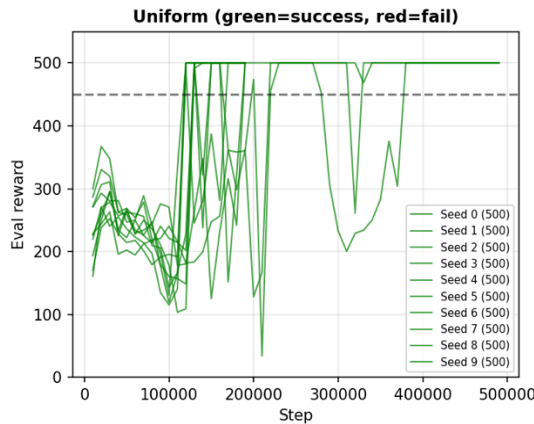
CRITICAL INSIGHT: UNIFORM PATTERN

Every parameter shows the SAME trend:
Strong prioritization → WORSE performance
Weak prioritization → BETTER performance

Key Findings & Interpretation (Major Discoveries)

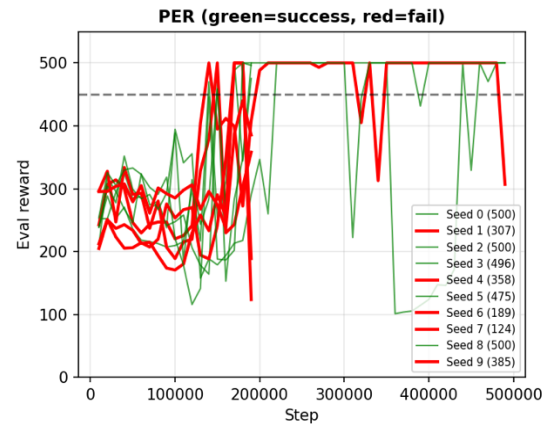
Finding 1: Uniform Replay Dominates

- Perfect Performance on CartPole (500 ± 0)
- Best on MountainCar despite not solving
- Zero failures across 20 runs
- ZERO hyperparameters to tune



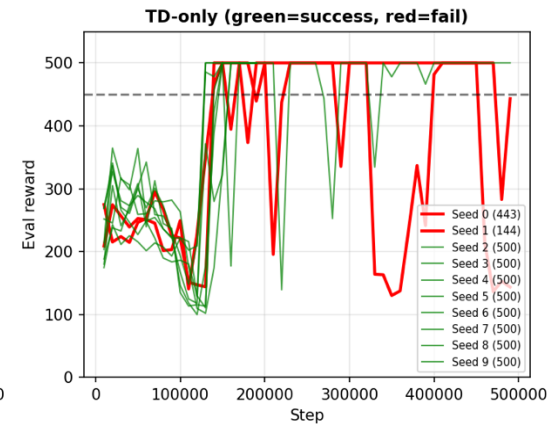
Finding 2: Prioritization - Instability

- High variance across seeds ($\sigma = 112$ -139)
- Frequent catastrophic failures (20-40%)
- Priority collapse on sparse rewards
- Exploration death on MountainCar



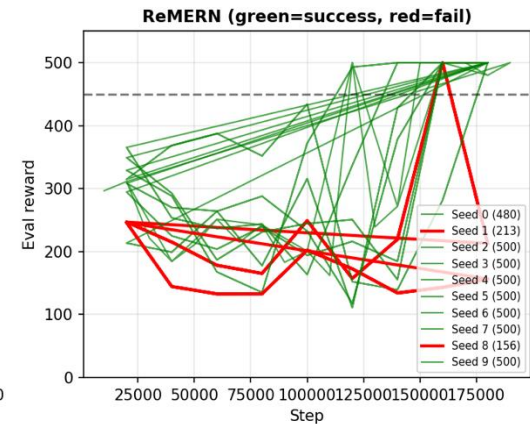
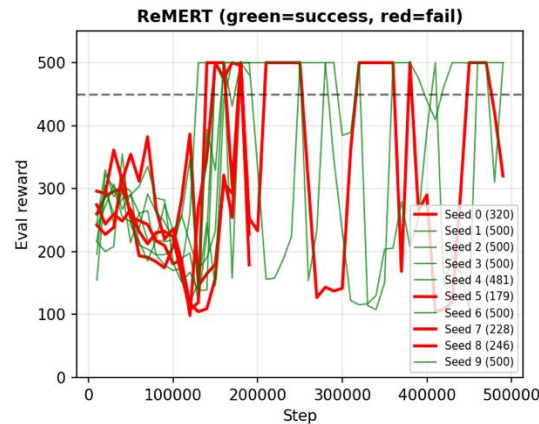
Finding 3: Complexity $\neq$ Performance

- HTD alone (459) > Full-RMER (395)
- Error Network provides minimal gain
- More components = more failure modes
- Simple methods more robust



Why Does Prioritization Fail?

1. Priority Collapse: Buffer fills with high-priority failures, successful experiences deprioritized
2. Exploration Death: Biased sampling reduces effective diversity, ϵ -greedy exploration insufficient
3. Hyperparameter Sensitivity: α , β , temperature require careful per-environment tuning
4. Task Mismatch: Benefits proven on MuJoCo (continuous, complex) don't transfer to discrete control



Comparison With Original Paper

RMER Paper (Liu et al., 2021) vs. Our Results

Aspect	Original Paper	Our Study
Environments	MuJoCO (HalfCheetah, Hopper, etc.)	CartPole MountainCar
Characteristics	• Continuous • Dense rewards • Complex dynamics	• Discrete • Dense/Sparse • Simple
Training	1,000,000 steps	200,000 steps
Results	RMER > PER > Uniform	Uniform > TD-only > PER $\approx$ RMER

Conclusions & Future Work

✓ Faithful Implementation

- Complete RMER reimplementation (all 3 components)
- Both ReMERT and ReMERN variants working
- Critical bug discovery and systematic debugging
- 100+ experimental runs with rigorous evaluation

✓ Surprising Empirical Findings

- Uniform replay dominates on both environments
- Prioritization introduces instability (20-40% failures)
- Simpler methods (TD-only) > complex (Full-RMER)
- Null results challenge field assumptions

✓ Scientific Contributions

- Environment-specific behavior documented
- Failure modes identified (priority collapse, etc.)
- Importance of rigorous evaluation (10 seeds minimum)
- Negative results have high value

Future Work :

- Extended Environments
- Method Improvements
- Theoretical Analysis
- Algorithmic Advances

Thanks For The Attention

Questions Welcome